OpenClaw SLAM Mapping And Navigation

OpenClaw SLAM Mapping And Navigation

1. Course Content
    Learning Objectives
2. Preparation
3. Recording the Map Mapping File
4. OpenClaw Intelligent Navigation
5. Source Code Analysis
    5.1 Navigation — Navigation Function
    5.2 PreciseAdjustPosition — Precise Position Adjustment
6. Parameter Debugging
    6.1 Navigation Mode Switching Parameters

1. Course Content

In complex real-world scenarios, robot navigation faces challenges at three levels:

1. Semantic place-name mapping (giving names to coordinates):
   Humans are used to saying *"go to the shipping area"* and *"go to the charging pile"*, while the underlying algorithm only recognizes spatial coordinates (X, Y, Z, Yaw). The system uses the `RecordMapLocation` tool to name and persistently save the current car pose with one click, just like putting "house numbers" on each shop in a mall.
2. Dual-mode navigation switching (highway vs. indoor slow walking):
    - Road network navigation: like "city main roads and elevated dedicated lanes". In open long-distance areas, the robot strictly drives at high speed along the pre-calibrated skeleton road network. The path is clear, it never gets lost, and it is extremely resistant to dynamic interference;
    - Ordinary autonomous navigation Nav2: near short-distance target points, it autonomously avoids obstacles and approaches through the local costmap.
    - The system uses `0.4 meters` as the critical point: long-distance travel uses the elevated road network, and approaching the station uses short-distance alignment, with the two seamlessly and smoothly connected!
3. **Precise positioning fine-tuning (PreciseAdjustPosition)**:
   After traveling tens of meters over a long distance, the chassis inevitably produces a drift error of a few centimeters due to tiny wheel slippage. In order for the robotic arm to accurately grasp the block on the tabletop, the car automatically starts ArUco visual relocalization after reaching the endpoint, and combines it with a PID controller to perform millimeter-level and angle-level precise alignment.

Learning Objectives

- Learn to use the RecordMapLocation tool to record a map mapping file and establish the correspondence between place names and coordinates.
- Understand the switching logic between road network navigation and ordinary navigation (Nav2).
- Master the PID control principle and parameter tuning methods of PreciseAdjustPosition precise position adjustment.

- Understand the application of visual relocalization (ArUco Tag) in improving navigation precision.

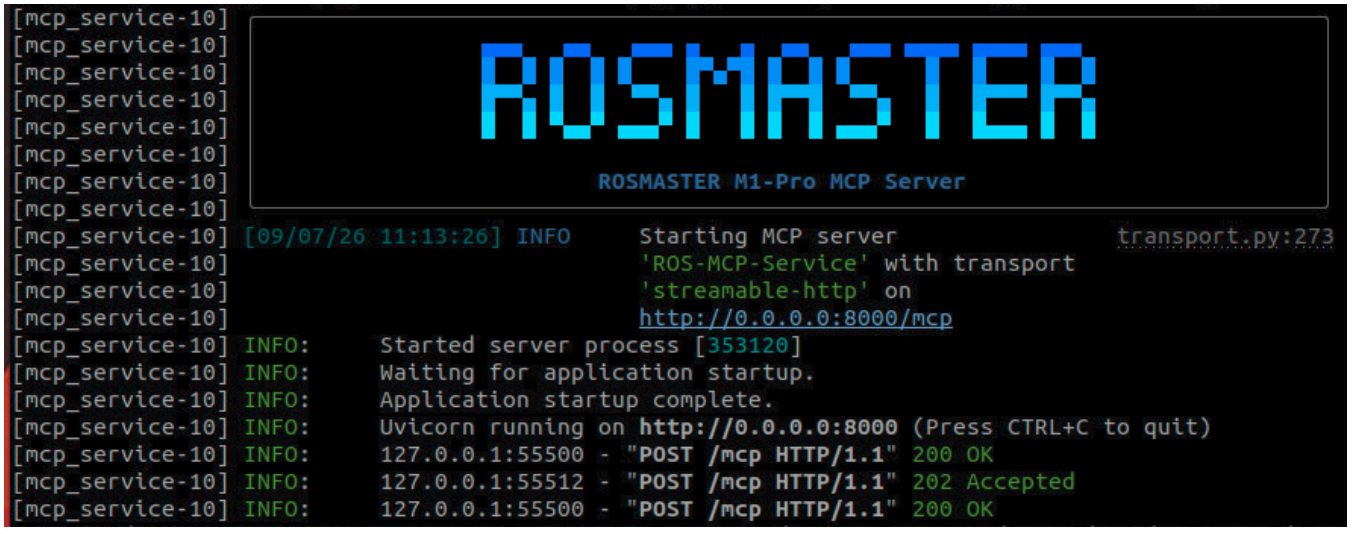
2. Preparation

> **Important**
>
> - OpenClaw's navigation control function requires first building a grid map, a pose map, and an annotated road network file. For details, refer to the tutorial [Road Network Planning and Navigation — Building a Pose Map and Road Network Annotation].
> - If you need to use the visual relocalization function, you also need to annotate the global position of the ArUco marker in the map. For detailed operations, refer to the tutorial [Aruco Marker Visual Relocalization Navigation] or [OpenClaw Open Transport Sandbox Map — Visual Relocalization Marker].

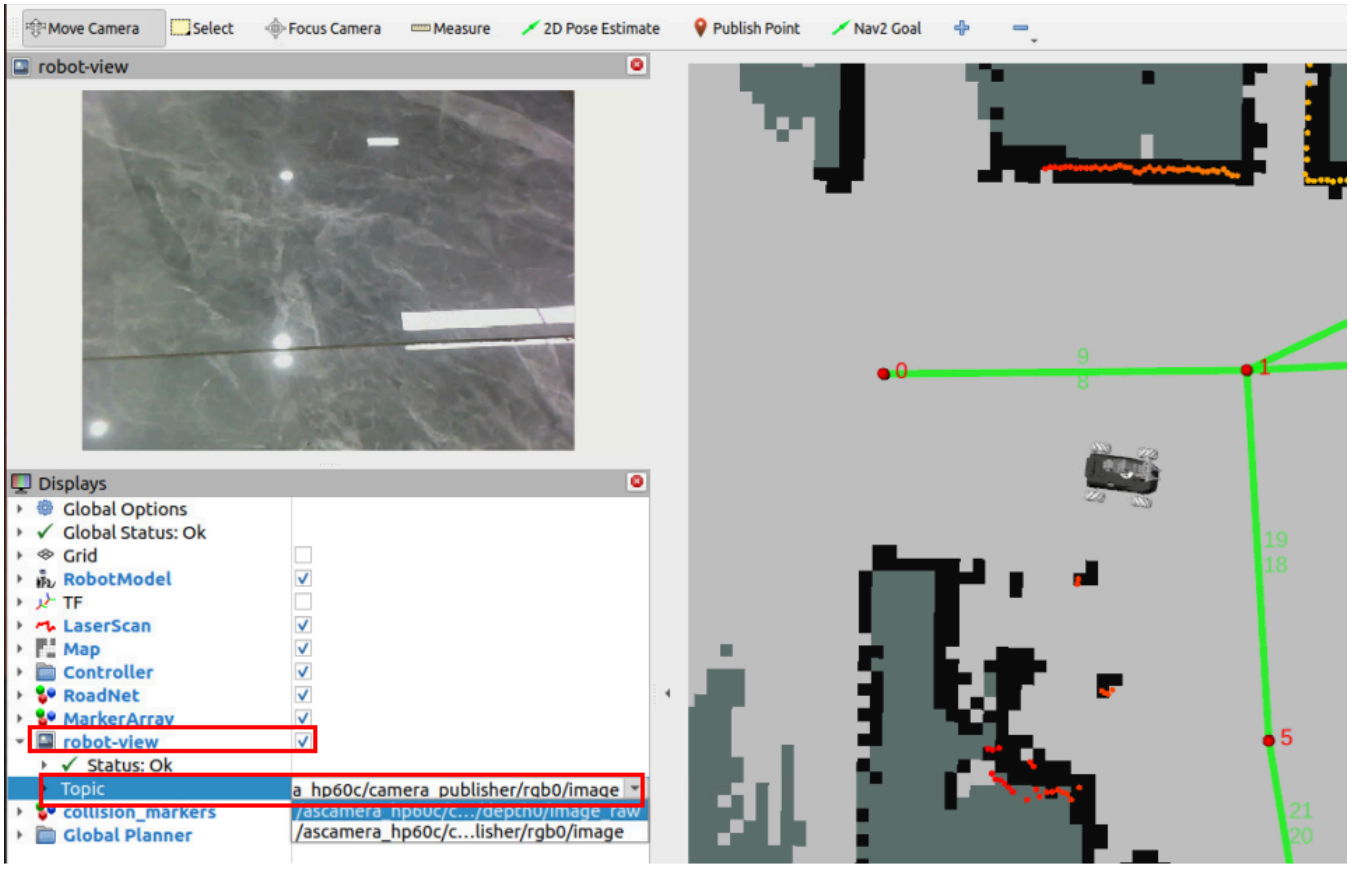
- Start nodes such as odometry, tf, robotic arm assist, and camera. If you need to enable the visual relocalization function, you need to add the startup parameter `relocalization:=true`. The demonstration here uses not starting relocalization as an example.

```
ros2 launch yahboomcar_bringup mcp_service.launch.py use_camera:=true use_nav:=true
```

> **Note**
>
> To balance the performance and positioning precision of different mainboards, the default localization algorithm is slam_toolbox, but you can switch among amcl, cartographer, and slam_toolbox through the parameter `localization_mode:=<xxx>`. Positioning precision and CPU usage: cartographer > slam_toolbox > amcl. For Raspberry Pi 5 (4GB) and RDK X5 mainboard users, if you feel lag during comprehensive navigation tasks, you can switch the localization mode to amcl (localization_mode:=amcl) to reduce CPU and memory pressure.



- If you need to view the camera image, select the camera topic from the Topic drop-down in the robot_view panel on the sidebar.

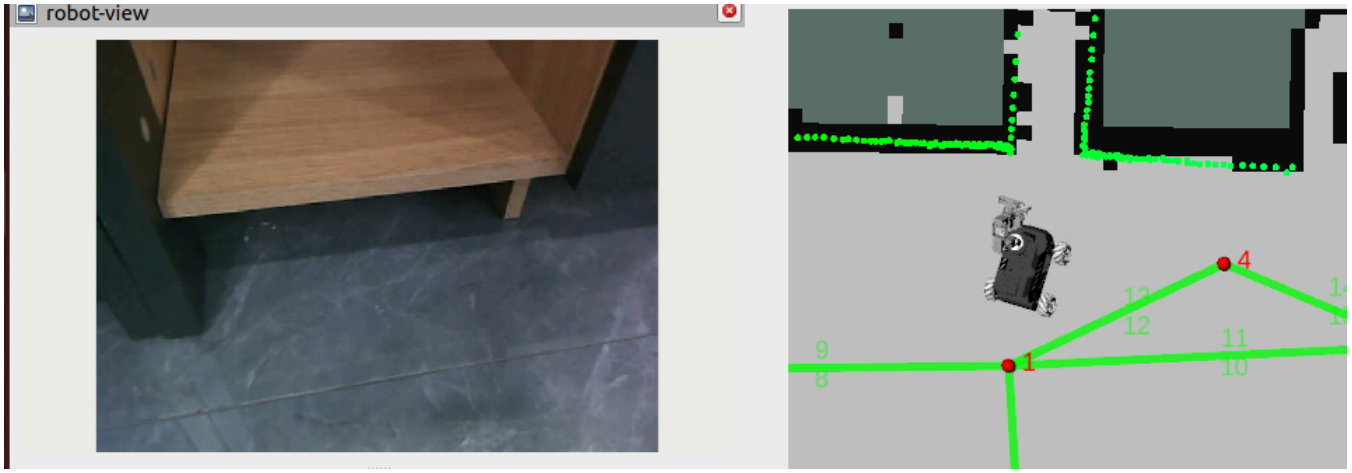


3. Recording the Map Mapping File

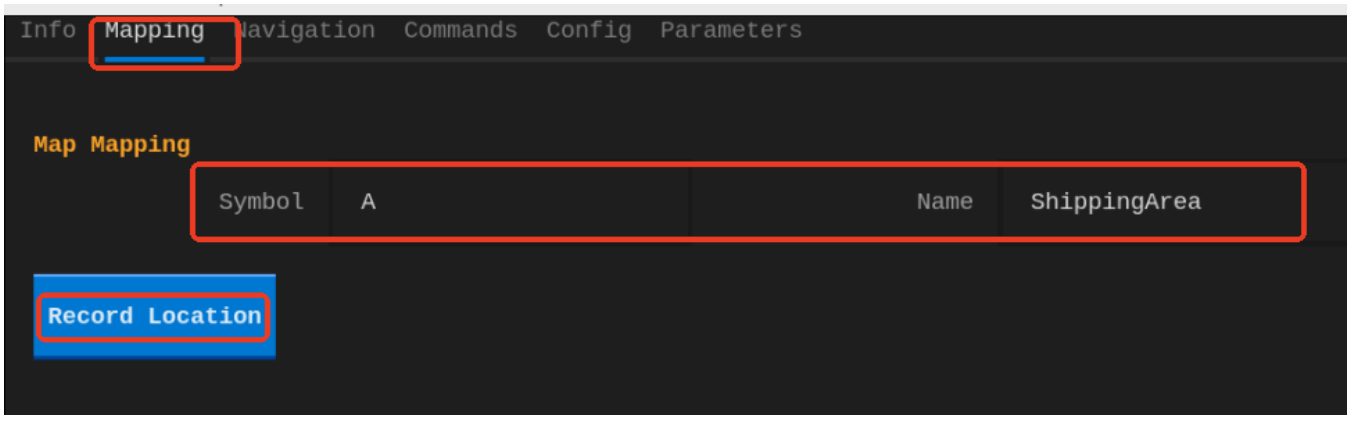
> **Tip**
>
> - You can use the robot_control CLI tool or directly ask OpenClaw to record the position and symbol through interaction with OpenClaw. The demonstration below uses recording the map mapping file with the CLI tool as an example.

Tool	Parameter	Function
RecordMapLocation	`--name`, `--symbol`	Save the current position to the map mapping file

- Move the car to the semantic position that needs to be recorded (any method works: keyboard, joystick, Nav2 goal, or transport).



- Choose either the RobotManager interface operation or the CLI operation below.
- **Method 1:** Fill in the symbol and semantic location, and click to record the position.



- A terminal will pop up in the VNC screen executing the position recording command. Press Enter to close it after completion.

```
Current location: x=0.521, y=0.652, z=-0.0
Symbol 'A' already exists, overwriting existing record
Location recorded successfully: A - Shipping Area

Press Enter to close.
```

- Method 2: Use the robot_control CLI tool to record the position of the car's **base_link** in the map **map**.

```
robot_control call-tool RecordMapLocation --name ShippingArea --symbol A
```

- --name specifies the name of the record point; --symbol specifies the symbol of the record point, making it easy for OpenClaw to recognize and distinguish.

```
jetson@yahboom:~$ robot_control call-tool RecordMapLocation --name Shipping Area --symbol B
{
  "execution_result": "success",
  "reason": ""
}
```

- If the record succeeds, the content shown above is displayed. If the map mapping file already has a symbol with the same name, the previous symbol content is automatically overwritten.

```
Z=0.0
[mcp_service-1] [WARN] [1778069773.818263710] [mcp_service]: Symbol 'B' already exists, overwrit
ing existing record
[mcp_service-1] [INFO] [1778069773.851449462] [mcp_service]: Location recorded successfully: B -
Shipping Area
```

- View the map mapping file map_mapping.yaml. The file path is:

```
~/yahboomcar_ros2_ws/yahboomcar_ws/src/yahboomcar_bringup/param/map_mapping.yaml
```

- Viewing the map_mapping.yaml file shows the location just recorded.

```
jetson@yahboom:~$ cat $HOME/yahboomcar_ros2_ws/yahboomcar_ws/src/yahboomcar_bringup/param/map_mapping.yaml
A:
  name: Shipping Area
  position:
    x: 1.975
    y: 1.587
    z: 0.0
  orientation:
    x: 0.0
    y: 0.0
    z: 0.654
    w: 0.756
B:
  name: Loading Area
  position:
    x: 1.942
    y: -2.541
    z: 0.0
  orientation:
    x: 0.0
    y: 0.0
    z: -0.749
```

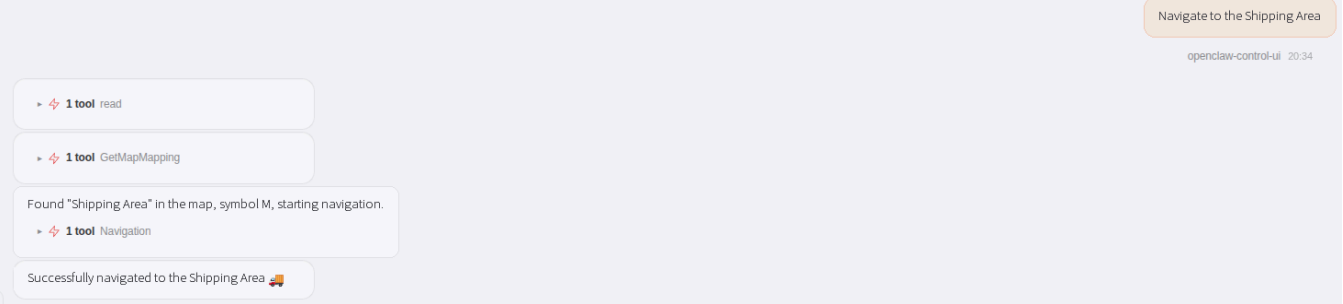
4. OpenClaw Intelligent Navigation

> **Tip**
>
> - You can write the command yourself according to your needs. The following is a case demonstration. Just choose any one of the interaction methods. The demonstration here uses the web-based WebChat as an example.

- Navigation function tools

Tool	Parameter	Function
GetMapMapping	None	Obtain the mapping between all place names and their corresponding identifiers
Navigation	`--location`	Navigate to the target position through the target point identifier

- After recording the mapping of semantic names, you can let the robot navigate to the specified position through the semantic name.



5. Source Code Analysis

- Source code path

```
~/yahboomcar_ros2_ws/yahboomcar_ws/src/multi_brains_pre/multi_brains_pre/action_controller.py
```

5.1 Navigation — Navigation Function

Function Description: Look up the corresponding coordinates from the map mapping file according to the target place name, then select the appropriate navigation mode (ordinary navigation or road network navigation) to execute the navigation task, and perform precise position adjustment after arrival.

- `Navigation`: The main navigation function, which looks up coordinates from the map mapping file according to the target place name and selects the appropriate navigation mode to execute.
- `yaml.safe_load(file)`: Reads the `map_mapping.yaml` map mapping file and parses the correspondence between all place-name symbols and coordinates.
- `for symbol, data in map_data.items()`: Iterates over the mapping table, supporting matching the target place by symbol name or the `name` field.
- `_get_current_map_pose()`: Obtains the robot's current pose in the map through TF transformation (`map → odom → base_link`).
- `distance < self.nav_mode_threshold`: Determines the navigation mode; if the distance is less than the threshold (default 0.8m), Nav2 ordinary navigation is used; otherwise, road network navigation is used.
- `NavigateToPose.Goal()`: Constructs the Nav2 navigation goal message, containing the position and orientation of the target position.
- `RoadNetToGoal.Goal()`: Constructs the road network navigation goal message; `use_start=False` means navigating from the current position.
- `self.relocalization`: The visual relocalization switch. If enabled, ArUco marker detection is started after navigation arrival to assist fine-tuning.
- `_control_aritag_detect(sign='enable'/'disable')`: Controls the start and stop of the ArUco marker detection node. If relocalization is enabled, the global localization is first corrected through relocalization, and then precise adjustment is performed.
- `_precise_adjust(goal_msg.pose)`: Calls the precise position adjustment function after navigation arrival, to eliminate accumulated errors.

```
    def Navigation(self, location: str) -> list[bool,str]:
        '''Use road network navigation'''
        location = location.strip("'\"")
        try:
            with open(self.map_mapping_file, 'r', encoding='utf-8') as file:
                map_data = yaml.safe_load(file) or {}

            target_data = None
            for symbol, data in map_data.items():
                if symbol == location or (isinstance(data, dict) and data.get('name') ==
location):
                    target_data = data
                    break
            if target_data is None:
                error_info = f"The map mapping file has no coordinate record for
'{location}'."
                self.get_logger().error(error_info)
                return [False,error_info]
        except Exception as e:
            self.get_logger().error(str(e))
            return [False,str(e)]

        goal_msg = PoseStamped()
        goal_msg.header.frame_id = "map"
        goal_msg.header.stamp = self.get_clock().now().to_msg()

        position = target_data.get('position', {})
        goal_msg.pose.position.x = float(position.get('x', 0.0))
        goal_msg.pose.position.y = float(position.get('y', 0.0))
        goal_msg.pose.position.z = float(position.get('z', 0.0))

        orientation = target_data.get('orientation', {})
        goal_msg.pose.orientation.x = float(orientation.get('x', 0.0))
        goal_msg.pose.orientation.y = float(orientation.get('y', 0.0))
        goal_msg.pose.orientation.z = float(orientation.get('z', 0.0))
        goal_msg.pose.orientation.w = float(orientation.get('w', 1.0))

        #calculate distance to target
        current_pose = self._get_current_map_pose()
        if current_pose is None:
            error_info = "Failed to retrieve the robot's current position in the map. Please
check if the Nav2 nodes are running properly. There may be an issue with the TF tree;
transforms between map -> odom -> base_link are missing."
            self.get_logger().error(Fore.RED + error_info + Fore.RESET)
            return [False, error_info]
        distance=math.sqrt((current_pose.position.x - goal_msg.pose.position.x) ** 2 +
                           (current_pose.position.y - goal_msg.pose.position.y) ** 2)

        if distance < self.nav_mode_threshold:
            #Close to the target point, use nav2 for navigation
            if not self.nav_to_pose_client.wait_for_server(timeout_sec=10.0):
                return [False, "Nav2 action server not available"]
            goal = NavigateToPose.Goal()
```

```
            goal.pose.header.frame_id = "map"
            goal.pose.header.stamp = self.get_clock().now().to_msg()
            goal.pose.pose.position.x = float(position.get('x', 0.0))
            goal.pose.pose.position.y = float(position.get('y', 0.0))
            goal.pose.pose.position.z = float(position.get('z', 0.0))
            goal.pose.pose.orientation.x = float(orientation.get('x', 0.0))
            goal.pose.pose.orientation.y = float(orientation.get('y', 0.0))
            goal.pose.pose.orientation.z = float(orientation.get('z', 0.0))
            goal.pose.pose.orientation.w = float(orientation.get('w', 1.0))
            send_goal_future = self.nav_to_pose_client.send_goal_async(goal)
            while rclpy.ok() and not send_goal_future.done():
                time.sleep(0.5)

            if not send_goal_future.done():
                return [False, "Failed to send navigation goal"]

            goal_handle = send_goal_future.result()
            if not goal_handle.accepted:
                return [False, "Navigation goal rejected"]

            result_future = goal_handle.get_result_async()

            while rclpy.ok() and not result_future.done():
                self.get_logger().debug("Waiting for navigation result...")
                time.sleep(0.5)

            if result_future.done():
                result = result_future.result().result
                if self.relocalization:
                    time.sleep(2.0) #TODO: why do this? wait for camera to stabilize after
navigation
                    self._control_aritag_detect(sign='enable')
                    time.sleep(2.0)
                adjust_result = self._precise_adjust(goal_msg.pose)
                if not adjust_result[0]:
                    self.get_logger().warn(Fore.YELLOW+f"Navigation arrived but precise
adjustment failed: {adjust_result[1]}"+Fore.RESET)
                if self.relocalization:
                    self._control_aritag_detect(sign='disable')# disable aritag
detection,decrease CPU usage
                return [True, "Navigation succeeded"]
            else:
                if self.relocalization:
                    self._control_aritag_detect(sign='disable')# disable aritag detection
                return [False, "Navigation failed"]

        else:
            # use road network navigation
            if not self.roadnet_nav_client.wait_for_server(timeout_sec=10.0):
                return [False, "RoadNet action server not available"]

            goal = RoadNetToGoal.Goal()
            goal.use_start = False
```

```
            goal.use_poses = True
            goal.goal = goal_msg

            self.get_logger().info(f"Sending roadnet navigation goal to '{location}'")

            send_goal_future = self.roadnet_nav_client.send_goal_async(goal)
            while rclpy.ok() and not send_goal_future.done():
                time.sleep(0.5)

            if not send_goal_future.done():
                return [False, "Failed to send roadnet navigation goal"]

            goal_handle = send_goal_future.result()
            if not goal_handle.accepted:
                return [False, "Roadnet navigation goal rejected"]

            result_future = goal_handle.get_result_async()

            while rclpy.ok() and not result_future.done():
                self.get_logger().debug("Waiting for roadnet navigation result...")
                time.sleep(0.5)

            if result_future.done():
                result = result_future.result().result
                if result.success:
                    if self.relocalization:
                        time.sleep(2.0)  # wait for camera to stabilize after navigation
                        self._control_aritag_detect(sign='enable')
                        time.sleep(2.0)
                    adjust_result = self._precise_adjust(goal_msg.pose)
                    if not adjust_result[0]:
                        self.get_logger().warn(Fore.YELLOW+f"Roadnet navigation arrived but
precise adjustment failed: {adjust_result[1]}"+Fore.RESET)
                    if self.relocalization:
                        self._control_aritag_detect(sign='disable')
                    return [True, f"Roadnet navigation to '{location}' succeeded"]
                else:
                    self.get_logger().warn(
                        Fore.YELLOW + f"Roadnet navigation to '{location}' failed: "
                        f"error_code={result.error_code}, {result.error_msg}" + Fore.RESET)
                    if self.relocalization:
                        self._control_aritag_detect(sign='disable')
                    return [False, result.error_msg]
            else:
                if self.relocalization:
                    self._control_aritag_detect(sign='disable')
                return [False, "Roadnet navigation failed"]
```

5.2 PreciseAdjustPosition — Precise Position Adjustment

Server source code path:

```
~/yahboomcar_ros2_ws/yahboomcar_ws/src/yahboomcar_bringup/yahboomcar_bringup/mcnamu_driver.py
```

Code Explanation:

- `_preciseAdjustActionCallback`: The Action server callback function for precise position adjustment, using a PD controller to achieve centimeter-level position alignment.
- `xy_kp / xy_kd`: XY position PD control gains; `kp` maps the distance error to linear velocity, and `kd` provides damping to suppress oscillation.
- `theta_kp / theta_kd`: Angle PD control gains; `kp` maps the angle error to angular velocity, and `kd` provides damping to suppress rotational oscillation.
- `consecutive_success >= required_success`: The arrival is confirmed only after both the position and angle errors are within tolerance for 3 consecutive detections, avoiding misjudgment caused by a single jitter.
- `_getMapPose()`: Obtains the current pose of `map → base_footprint` through TF; waits up to 10 times (5 seconds) before starting.
- `_odom_action_active = True`: Locks the chassis speed control authority to prevent external `cmd_vel` from interfering with the fine-tuning process.
- `dx / dy / pos_error`: Computes the position offset in the map coordinate system, `pos_error = √(dx² + dy²)`.
- `d_dx / d_dy / d_yaw_error`: Error rate of change (derivative term), used for the D component of PD control to provide damping.
- `phase = _PHASE_DRIVE / _PHASE_FINAL_YAW`: Two-stage control strategy — omnidirectional driving when the position is not reached, and in-place rotation to align the angle after the position is reached.
- `cos_th / sin_th`: Rotates the error in the map coordinate system into the body coordinate system, realizing independent front/back/left/right control of the omnidirectional chassis.
- `min_linear_speed / min_angular_speed`: Anti-dead-zone compensation; when the control value is too small to overcome static friction, the minimum speed is forcibly output.
- `precise_adjust_pos_tolerance / precise_adjust_angle_tolerance`: Position and angle tolerances; an error smaller than this value is regarded as arrival.

```
    def _preciseAdjustActionCallback(self, goal_handle):
        '''Action callback: omnidirectional precise adjustment to a target map pose (Mecanum)'''
        goal = goal_handle.request
        target_x = goal.target_pose.position.x
        target_y = goal.target_pose.position.y
        target_yaw = self._getYawFromQuaternion(goal.target_pose.orientation)

        self.get_logger().info(
```

```
            f"PreciseAdjust goal: x={target_x:.3f}, y={target_y:.3f}, yaw=
{math.degrees(target_yaw):.1f}°")

        # Gains (P + D)
        xy_kp = self.precise_adjust_xy_pid[0]        # distance -> linear speed
        xy_kd = self.precise_adjust_xy_pid[2]        # damping for linear oscillation
        theta_kp = self.precise_adjust_theta_pid[0]  # heading error -> angular speed
        theta_kd = self.precise_adjust_theta_pid[2]  # damping for angular oscillation

        consecutive_success = 0
        required_success = 3

        # Wait for TF listener cache to be established before starting control
        pose = None
        for attempt in range(10):
            if goal_handle.is_cancel_requested:
                goal_handle.canceled()
                result = PreciseAdjustPose.Result()
                result.success = False
                result.message = "Precise adjustment cancelled"
                return result
            pose = self._getMapPose()
            if pose is not None:
                break
            time.sleep(0.5)
        if pose is None:
            self.get_logger().error("Failed to get map->base_footprint TF after 10
attempts")
            goal_handle.abort()
            result = PreciseAdjustPose.Result()
            result.success = False
            result.message = "Failed to get map->base_footprint TF"
            return result

        phase = self._PHASE_DRIVE
        pos_error = 0.0
        yaw_error = 0.0
        prev_dx = 0.0
        prev_dy = 0.0
        prev_yaw_error = 0.0
        prev_time = time.time()

        # Suppress external cmd_vel during adjustment
        self._odom_action_active = True
        try:
            for _ in range(self.precise_adjust_max_iteration):
                # Cancel check
                if goal_handle.is_cancel_requested:
                    self.car.set_car_motion(0.0, 0.0, 0.0)
                    goal_handle.canceled()
                    result = PreciseAdjustPose.Result()
                    result.success = False
                    result.message = "Precise adjustment cancelled"
```

```
                return result

            pose = self._getMapPose()
            if pose is None:
                self.get_logger().error("Failed to get current pose from TF")
                self.car.set_car_motion(0.0, 0.0, 0.0)
                goal_handle.abort()
                result = PreciseAdjustPose.Result()
                result.success = False
                result.message = "Failed to get current pose from TF"
                return result

            current_x, current_y, current_yaw = pose

            # Position/orientation error in the map frame
            dx = target_x - current_x
            dy = target_y - current_y
            pos_error = math.hypot(dx, dy)
            yaw_error = compute_angle_diff(target_yaw, current_yaw)

            # Error derivatives for D term (damping)
            now = time.time()
            dt = now - prev_time
            prev_time = now
            if dt > 1e-6:
                d_dx = (dx - prev_dx) / dt
                d_dy = (dy - prev_dy) / dt
                d_yaw_error = (yaw_error - prev_yaw_error) / dt
            else:
                d_dx = 0.0
                d_dy = 0.0
                d_yaw_error = 0.0
            prev_dx = dx
            prev_dy = dy
            prev_yaw_error = yaw_error

            pos_reached = pos_error < self.precise_adjust_pos_tolerance
            angle_reached = abs(yaw_error) < self.precise_adjust_angle_tolerance

            # Publish feedback
            feedback = PreciseAdjustPose.Feedback()
            feedback.position_error = float(pos_error)
            feedback.angle_error = float(math.degrees(yaw_error))
            goal_handle.publish_feedback(feedback)

            if pos_reached and angle_reached:
                consecutive_success += 1
                if consecutive_success >= required_success:
                    self.car.set_car_motion(0.0, 0.0, 0.0)
                    msg = (f"Precise adjustment completed: pos_error={pos_error:.3f}m, "
                           f"angle_error={math.degrees(yaw_error):.1f}°")
                    self.get_logger().info(msg)
                    goal_handle.succeed()
```

```
                        result = PreciseAdjustPose.Result()
                        result.success = True
                        result.message = msg
                        return result
                else:
                    consecutive_success = 0

                # Phase transition: hold position and only align yaw once the point is
reached
                phase = self._PHASE_FINAL_YAW if pos_reached else self._PHASE_DRIVE

                if phase == self._PHASE_FINAL_YAW:
                    # In-place rotation to the target yaw
                    vx = 0.0
                    vy = 0.0
                    omega = theta_kp * yaw_error + theta_kd * d_yaw_error
                    omega = max(-self.precise_adjust_max_angular_vel,
                                min(self.precise_adjust_max_angular_vel, omega))
                    if angle_reached:
                        omega = 0.0
                    elif abs(omega) < self.min_angular_speed:
                        # Overcome the chassis static-friction dead-zone so it actually
turns.
                        omega = self.min_angular_speed if omega > 0 else -
self.min_angular_speed
                else:
                    # Omnidirectional drive: rotate map-frame error into the body frame
                    cos_th = math.cos(current_yaw)
                    sin_th = math.sin(current_yaw)
                    ex = dx * cos_th + dy * sin_th
                    ey = -dx * sin_th + dy * cos_th
                    # Derivatives in body frame
                    d_ex = d_dx * cos_th + d_dy * sin_th
                    d_ey = -d_dx * sin_th + d_dy * cos_th
                    vx = xy_kp * ex + xy_kd * d_ex
                    vy = xy_kp * ey + xy_kd * d_ey
                    # Clamp combined linear speed
                    speed = math.hypot(vx, vy)
                    if speed > self.precise_adjust_max_linear_vel:
                        scale = self.precise_adjust_max_linear_vel / speed
                        vx *= scale
                        vy *= scale
                    # Anti-stall dead-zone per axis (only while that axis still has error
left)
                    if abs(vx) > 1e-6 and abs(vx) < self.min_linear_speed and abs(ex) >
self.precise_adjust_pos_tolerance:
                        vx = self.min_linear_speed if vx > 0 else -self.min_linear_speed
                    if abs(vy) > 1e-6 and abs(vy) < self.min_linear_speed and abs(ey) >
self.precise_adjust_pos_tolerance:
                        vy = self.min_linear_speed if vy > 0 else -self.min_linear_speed
                    # Keep yaw aligned to the target while driving (with D damping)
                    omega = theta_kp * yaw_error + theta_kd * d_yaw_error
                    omega = max(-self.precise_adjust_max_angular_vel,
```

```
                                min(self.precise_adjust_max_angular_vel, omega))
                    if abs(omega) < self.min_angular_speed:
                        omega = self.min_angular_speed if omega > 0 else -
self.min_angular_speed

                self.car.set_car_motion(vx, vy, omega)
                time.sleep(0.1)  # 10Hz

            # Max iterations reached
            self.car.set_car_motion(0.0, 0.0, 0.0)
            msg = (f"Precise adjustment reached max iterations
({self.precise_adjust_max_iteration}). "
                   f"pos_error={pos_error:.3f}m, angle_error={math.degrees(yaw_error):.1f}
°")
            self.get_logger().warn(msg)
            goal_handle.abort()
            result = PreciseAdjustPose.Result()
            result.success = False
            result.message = msg
            return result
        finally:
            self._odom_action_active = False
            # Ensure robot stops
            self.car.set_car_motion(0.0, 0.0, 0.0)
```

6. Parameter Debugging

> **Important**
>
> Parameters can be adjusted through the RobotManager interface or by directly editing the file.

- Parameter file path:

```
~/yahboomcar_ros2_ws/yahboomcar_ws/src/yahboomcar_bringup/param/common.yaml
```

6.1 Navigation Mode Switching Parameters

```yaml
mcp_service:
  ros__parameters:
    # Navigation
    nav_mode_threshold: 0.8  # Navigation mode switching threshold, unit: meters
```

Parameter Description:

Parameter	Type	Meaning	Default value	Adjustment suggestion
`nav_mode_threshold`	float	Navigation mode switching distance threshold (meters)	0.8	If the navigation distance is less than the threshold, ordinary navigation is used; if greater than this threshold, road network navigation is used